HONG KONG BAPTIST UNIVERSITY

Audio and Speech Processing

Practical Assignment I

Introduction

This practical assignment is designed to assess students' understanding and programming skills of conducting Machine Learning with Audio Signals. Students will be tested on their replication and implementation of building a basic pipeline, including data preprocessing, analysis, and feature extraction (engineering). This assignment is shared by both undergraduate and postgraduate students. Each task contains a core part (following the code template) and clearly marked **TODO** sections that you must complete yourself, plus an **[Extended]** sub-part that asks for a somewhat deeper, more quantitative analysis so that the assignment remains appropriately challenging for postgraduate students while staying manageable for undergraduates. Bonus scores will be provided to students for their innovative programming on the extra tasks with additional feature construction. Students are also required to report results by showing critical thinking in interpreting the results.

Data

Like the sample data demonstrated in the lab, two sets of data are provided. Each student will work on the data according to the number of the last digit in your Student ID. Please refer to Table 1 in Appendix A to find the data to work on.

Program Code Template

An updated program code template is provided as **"Audio_1_Template.ipynb"**. The template follows the same overall pipeline shown in the lab, but a number of lines have been intentionally left as `# TODO` for you to complete, rather than being fully worked examples. You should complete every TODO in order for each cell/task to run correctly. Google Colab and PyCharm can be the platform for you to execute the programming.

Tasks

- **Basic Task:** Complete the setup TODOs (loading your assigned pair of audio files) and run the demo pipeline in the template file on your given subset of data. Report the implementation results in your .docx file, with basic descriptions of the results.
- **Task 1 — Sampling frequency effects.** Complete the TODOs to resample your audio to sr = 44100 Hz and regenerate the waveform and spectrogram. Comment on the difference between the figures and report how it changes the audio playback effect. Include the changed waveform and spectrogram figures in the report.
    - **[Extended]** Compute the frequency resolution (delta_f = sr / n_fft) for the original and the 44100 Hz signals using the default n_fft, present the values in a small table, and explain in your own words why a higher sr improves frequency resolution for a fixed n_fft, and what trade-off this involves.
- **Task 2 — Spectrogram features.** Complete the TODOs to build a reusable spectrogram function, then compare the spectrogram features when the window lengths are 128, 256, and

512. Record these spectrogram images and their sizes. Fix the window length to 256 and change the hop_length to ½ and ¼ of it, again recording the images and sizes. Plot the figures with your chosen musical instrument(s) and report how these changes affect the visualization.

- **[Extended]** For every configuration tested above, compute both the time resolution (delta_t = hop_length / sr) and the frequency resolution (delta_f = sr / n_fft) and present them together in one summary table. Use this table to quantitatively discuss the time–frequency resolution trade-off, rather than describing it only qualitatively.

- **Task 3 — Data augmentation.** Complete the TODOs to compare the audio effect and spectrogram when librosa.effects.trim is applied to the signal (plot original vs. trimmed waveforms and spectrograms side-by-side). Report your findings.

- **[Extended]** Apply two further augmentation techniques to the trimmed signal: time-stretching (librosa.effects.time_stretch) at rate = 0.8 and rate = 1.2, and pitch-shifting (librosa.effects.pitch_shift) by +2 semitones. Compare the resulting array shapes and briefly discuss why time-stretching changes the signal length while pitch-shifting does not, and how each technique could help when training an audio classification model.

- **Task 4 — MFCC features.** Complete the TODOs to build a reusable MFCC extraction/plotting function, then compare the MFCC features when the number of MFCCs is 12, 24, and 32. Record the sizes of the resulting features and discuss the difference in the report.

- **[Extended]** Using n_mfcc = 24, additionally compute the delta and delta-delta MFCC coefficients (librosa.feature.delta) and stack them with the static MFCCs into one combined feature matrix. Report the shape of each component and discuss what information the delta / delta-delta coefficients add on top of the static MFCCs, and why this combined representation is commonly used in speech/audio recognition systems.

- ***Extra/bonus Task 1:** Re-express each task's code above as function format, extending the functions started in Tasks 2 and 4 to Tasks 1 and 3 as well. Clearly specify the inputs and outputs of every function (docstrings are expected). Report your results in the report and submit the code as **"code_improved.ipynb"**.

- ***Extra/bonus Task 2:** Perform audio dimensionality reduction with PCA. Build a spectrogram-frame feature matrix from your two assigned audio files, standardize it, and perform PCA (sklearn.decomposition.PCA) for dimensionality reduction down to 2 components. Check the scikit-learn documentation to call the function correctly. Plot the 2D projection (coloured by which file each frame came from) and report the explained variance ratio. Discuss the implementation results, e.g. how well the two instruments separate in the reduced space.

Tasks:

- **Tasks and Results:** Report the results of each task with point-wise subsections, e.g., 1.1, 1.2, 1.3 … (including the Extended sub-parts, e.g., 1.4). Provide results descriptions and interpret them with proper analysis. Showing your understanding of audio features — including the quantitative Extended analysis — will make the report outstanding.
- **Summary:** Summarize interesting findings observed across the tasks.
- **Discussion:** Discuss the interesting and difficult aspects of the implementation, including the Extended sub-parts.
- **Conclusion:** Conclude the report and provide suggestions for implementation.

Submission

1. Submit ONE .ipynb file containing all your program codes for the core tasks (Basic Task and Tasks 1–4, including all TODOs completed and Extended sub-parts). Rename the .ipynb file to your name, e.g. ChanTaiMan.ipynb.

2. Submit the bonus problem files (Bonus Task 1 and Bonus Task 2) separately as code_improved.ipynb.

3. Submit your written report as a .docx file, with results organised in point-wise subsections as described above.

Appendix A

You should work on one of the corresponding subset data as listed in Table 1.

Last digit of student ID	**Name of subset data**
0-4	10Piano.wav, 10Violins.wav
5-9	10String.wav, 10Bass.wav

Table 1 - Mapping of Student ID and Subset Number.